

Technology Stack

Date	16 September 2026
Team ID	SWTID-2026-2825
Project Name	AI WeatherWise
Maximum Marks	2 Marks

Technology Stack Details

AI WeatherWise is a headless RESTful backend; the "client" during development and grading is an API tool (Thunder Client / Postman) rather than a browser UI.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	Thunder Client (VS Code) & Postman — used as the request/response client; no browser UI was built for this cycle	Project scope for 15–29 Sept 2026 is a backend/API service; API clients are sufficient to exercise and grade every endpoint via <code>postman_collection.json</code>
2	Backend / Server-Side	Node.js + Express.js	Non-blocking I/O suits the I/O-bound weather and AI API calls; minimal boilerplate for fast REST route development; whole team already proficient in JavaScript
3	Database / Data Storage	MongoDB with Mongoose ODM	Schema-flexible NoSQL store fits the evolving User/Location documents; Mongoose adds field validation, defaults and relations on top of raw MongoDB
4	Cloud / Hosting / Deployment	Local Node.js runtime for the grading window; fully <code>.env</code> -driven configuration (PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY) so it is cloud-deploy-ready	Keeps the grading environment simple and reproducible while leaving a clear path to Render/Railway/Azure deployment without code changes
5	Version Control & CI/CD	Git & GitHub – github.com/SafiyahArif/AI_Weather_TN_Skills	Single source of truth for the 4-member team; commit history traces each Epic/Sprint
6	Third-Party APIs / Other Tools	OpenWeatherMap API (live weather), Google Gemini AI – <code>@google/genai</code> (natural-language insights), <code>jsonwebtoken</code> + <code>bcryptjs</code> (auth/security), <code>dotenv</code> , <code>cors</code> , <code>multer</code> , <code>nodemon</code> (dev)	Each library enables one specific, named feature of the solution — live data, generative insight, secure auth, config isolation, and cross-origin support